



VRIJE  
UNIVERSITEIT  
BRUSSEL



# LOOP CLOSURE DETECTION

Based on: *Generalized Detection and Merging of Loop Closures for Video Sequences*

Quentin Gerard Hacourt, Nerissa Janssens, Noémi Veres

December 21, 2025

# Contents

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>Summary</b>                 | <b>1</b> |
| <b>2</b> | <b>Implementation</b>          | <b>1</b> |
| 2.1      | Data Acquisition . . . . .     | 1        |
| 2.2      | Preprocessing Steps . . . . .  | 1        |
| 2.3      | Algorithm . . . . .            | 1        |
| <b>3</b> | <b>Results</b>                 | <b>3</b> |
| 3.1      | Quantitative Results . . . . . | 3        |
| 3.2      | Qualitative Results . . . . .  | 3        |
| <b>4</b> | <b>Discussion</b>              | <b>6</b> |
| <b>5</b> | <b>Reproducibility Check</b>   | <b>6</b> |

# 1 Summary

This project aims to implement the loop closure detection method proposed by Klopschitz et al. in [1]. Our method processes images extracted from video sequences to detect overlaps between the camera trajectory. Such techniques are useful in simultaneous localization and mapping (SLAM) applied in mobile robotics or autonomous driving, where we need to detect revisited places for building maps and ensuring navigation accuracy.

## 2 Implementation

The implementation is divided into the following main steps:

- constructing a visual similarity matrix (VSM) based on image features
- obtaining dissimilarity matrix  $D$  from the VSM
- filling in a cost matrix  $H$
- extracting optimal paths from  $H$

### 2.1 Data Acquisition

The dataset used for the experiment presented in this work is a image sequence of the ETH3D Stereo Benchmarks dataset [2]. More precisely this work presents an experiment performed on "old computer" images.

### 2.2 Preprocessing Steps

Before running our algorithm on the dataset, the dataset is split in two sequences (with some overlap between the two sequences).

### 2.3 Algorithm

The **Visual Similarity Matrix (VSM)** is constructed based on the extracted features and each entry  $VSM(x, d)$  represents the similarity between image  $x$  of the first sequence and image  $d$  of the second. In our case, we considered the same sequence as we want to detect loop closure within a given video. For the optimization step, this is converted into a **Dissimilarity Matrix**  $D = -VSM$ , which contains normalized values. This VSM is built by computing similarity scores with `pyvisim` library [3].

Once the dissimilarity matrix is obtained, one can apply **Scanline Optimization** to find the optimal path of correspondence between the two sequences. This method is described in [4]. The algorithm constructs a cost matrix  $H$  where  $H(x, d)$  represents the minimum accumulated cost of an alignment path ending at  $(x, d)$ . The relation is defined as:

$$H(x, d) = \min(0, D(x, d) + h(x, d)) \quad (1)$$

where  $D(x, d)$  is the dissimilarity cost of the current image pair, and  $h(x, d)$  represents the minimum cost from the previous row (best previous path), calculated as:

$$h(x, d) = \min_{d'} (H(x - 1, d') + \lambda V(d, d')) \quad (2)$$

Here,  $\lambda$  is a regularization weight,  $d'$  is an image in sequence 2 for the previous step (the column in the previous row) and  $V(d, d')$  is the transition cost, defined to favor diagonal movement (cost being zero in this case):

$$V(d, d') = \min(|(d + 1) - d'|, |(d - 1) - d'|) \quad (3)$$

In our implementation  $\lambda$  was set to 1.2. We found out that a lambda that is too low, for example see figure 1 leads to excessive jumping around therefore an incoherent path.

By using  $\min(0, \dots)$  in Eq. (1) we reset the accumulated cost if the path becomes bad.

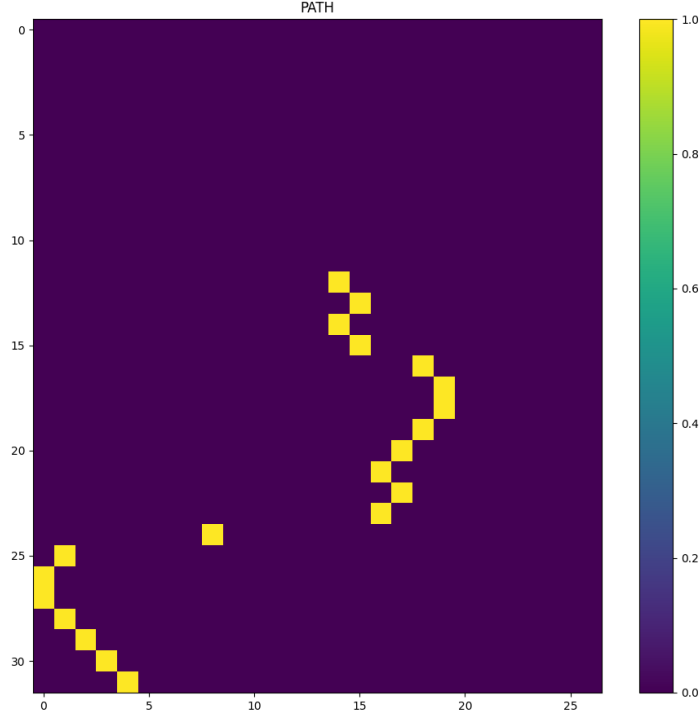


Figure 1: Path plot,  $\lambda$  value of 0.2 leads to excessive jumping around.

The final step of our algorithm is to "normalize" the H-matrix.  $score(x, d)$  is multiplied by the fraction of  $width(path, x) * width(path, d) / ||path||^2$ ; where path is the previously calculated path (Figure 5(a)) and  $width(path, x)$  equals the amount of images in the path that belongs to sequence x. This step is useful when trying to distinguish sequence matches of many frames at the same location and the actual sequence matches we are looking for, those further apart in distance, that are necessary for loop detection.

Once we have the normalized  $H$  matrix, the optimal loop closure is detected by finding the minimum value in the entire matrix (representing the best ending of a path) through backtracking.

The backtracking algorithm works as defined below (algorithm 1). What it comes down to is that it iteratively finds each next step by looking for the lowest score and adding weight  $\lambda * v\_cost$  to factor an extra cost of jumping around. Notice how it can either stop when the

value for *current\_x* becomes negative or if the *score(x, d)* value is greater or equal to  $-6.5$ . The latter check is useful to make the path stop when the overlap is in the end of the second sequence and the start of the first one (so on the lower left corner in our VSM).

---

**Algorithm 1** Backtrack the optimal path in cost matrix  $H$

---

```

BacktrackPath( $H, \lambda$ )
1:  $n, m \leftarrow \text{shape}(H)$ 
2:  $(\text{current\_x}, \text{current\_d}) \leftarrow \text{index of min}(H)$ 
3:  $\text{path} \leftarrow [(\text{current\_x}, \text{current\_d})]$ 
4: while  $\text{current\_x} > 0$  do
5:   if  $H[\text{current\_x}, \text{current\_d}] \geq -6.5$  then
6:     break
7:   end if
8:    $\text{prev\_x} \leftarrow \text{current\_x} - 1$ 
9:    $\text{min\_cost} \leftarrow \infty$ 
10:  for  $\text{prev\_d} = 1$  to  $m$  do
11:     $v\_cost \leftarrow \text{regularisation\_cost}(\text{current\_d}, \text{prev\_d})$ 
12:     $\text{total\_cost} \leftarrow H[\text{prev\_x}, \text{prev\_d}] + (\lambda \times v\_cost)$ 
13:    if  $\text{total\_cost} < \text{min\_cost}$  then
14:       $\text{min\_cost} \leftarrow \text{total\_cost}$ 
15:       $\text{best\_prev\_d} \leftarrow \text{prev\_d}$ 
16:    end if
17:  end for
18:   $\text{current\_d} \leftarrow \text{best\_prev\_d}$ 
19:   $\text{current\_x} \leftarrow \text{prev\_x}$ 
20:   $\text{path.append}((\text{current\_x}, \text{current\_d}))$ 
21: end while
22: return  $\text{reverse}(\text{path})$ 

```

---

## 3 Results

### 3.1 Quantitative Results

We evaluated the runtime efficiency of the pipeline. The most computationally intensive stage was the construction of the Visual Similarity Matrix (VSM). The initial implementation involved comparing the complete video sequence against itself and generating a square matrix, which required approximately **3 hours and 40 minutes**. This confirmed that feature extraction and pairwise comparison are the primary bottlenecks in the system. By splitting the dataset into smaller, overlapping subsequences (as described in Section 2.2), we reduced the matrix dimensions for individual runs, resulting in a **decrease** in processing time.

### 3.2 Qualitative Results

Figure 2 shows two images which look similar to the human eye and we obtain a high similarity score accordingly. In contrast, in Figure 3 we show two images with two different views, getting a negative similarity score with a smaller absolute value. In case of images, the score 1 would show a perfect match, while 0 means that there is no meaningful similarity between images. The score

-1 is quite rare for images as it indicates that the features that define one image are strongly anti-correlated with those of the other. A score of -0.5 rather shows that a contrast is detected between images.



Figure 2: Two images picked from the dataset with similarity score 0.71



Figure 3: Another pair of images from the same dataset with similarity score of  $-0.5$

Given these similarity scores, we can compute these for all image pairs of both sequences, resulting in following (Dis)similarity Matrix.

Figure 4 illustrates the dissimilarity matrix and the optimized cost matrix  $H$  used for loop detection.

In Figure 4(a), dark purple regions indicate high similarity (values approaching -1.0), while yellow regions indicate dissimilarity (values  $> 0$ ). We can detect a dark line in the bottom-left corner (rows 26–30 intersecting with columns 0–5), suggesting that the end of the first sequence resembles the beginning of the other one.

Figure 4(b) presents the **Accumulated Cost Matrix** ( $H$ ) resulting from the Scanline Optimization. Here, each entry reflects the cumulative cost at each position. The matrix is initialized with the first row of  $D$ , and subsequent rows are computed by minimizing the sum of previous costs and a regularization penalty, followed by a clipping operation. The resulting matrix reveals patterns of the accumulated penalties: as the optimization proceeds, a degrade pattern is obtained.

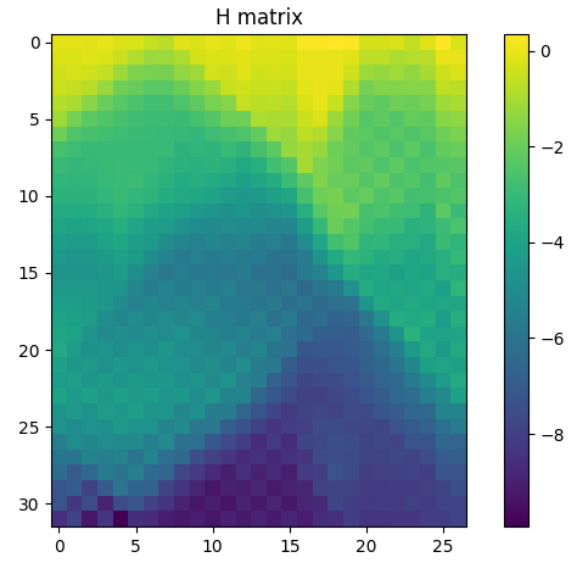
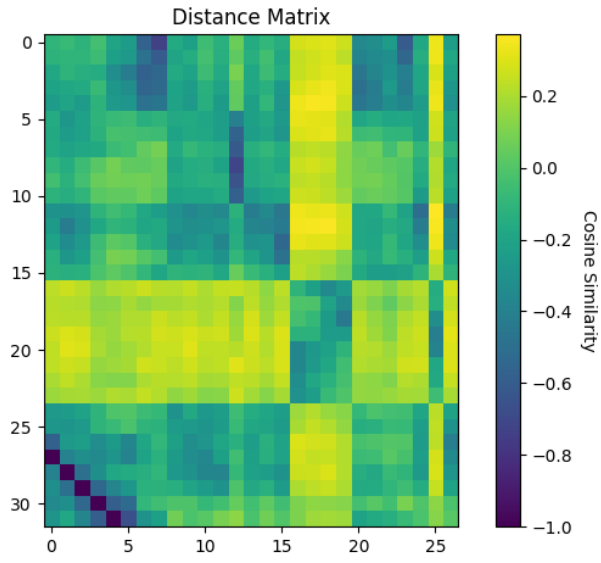


Figure 4: Dissimilarity Matrix (left) computed with  $D = -S$  (similarity matrix as computed before), H-matrix (right)

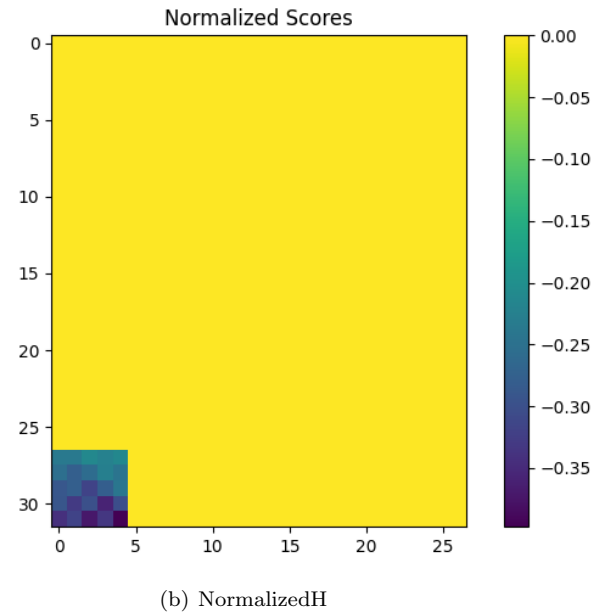
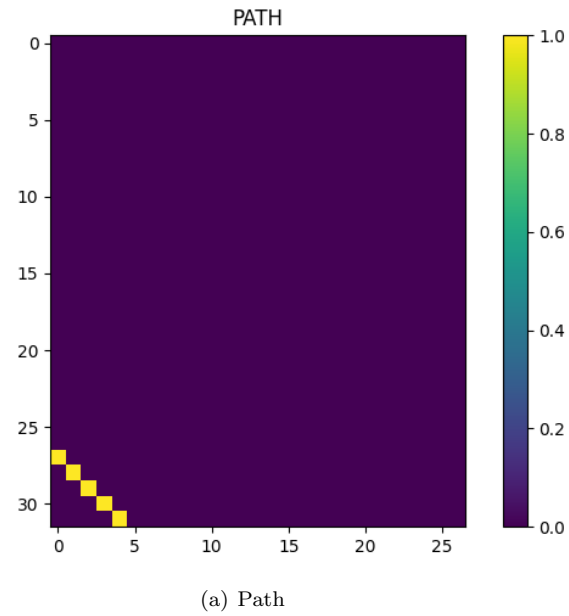


Figure 5: Path plot (left), represents the found path after backtracking from scanline optimization, Normalized  $H_n$  matrix (right)

## 4 Discussion

The results in Section 3 show that our algorithm is able to detect loop closures. The difference between the Distance Matrix (Figure 4(a)) and the Accumulated Cost Matrix  $H$  (Figure 4(b)) highlights the main advantage of this method. The Distance Matrix shows scattered dark spots, which represent individual frames that look similar. However, relying on single images is unreliable because different places can often look alike. This is why the  $H$  matrix is looking for a *sequence* of matches, founding a continuous path where frames matched over time. By this way, we have real loop closure detection and not just a visual similarity.

Our results show large yellowish regions in the  $H$  matrix where the cost is close to 0. This means the algorithm "gave up" on sections of the video that did not match. The algorithm "waits" in the 0 cost state until it finds strong matches at which point it starts accumulating a negative score and form a "valley" as seen in Figure 4(b).

**Limitations:** Our method relies on SIFT features remaining consistent. In situations with drastical lighting changes or moving objects, the raw similarity scores might become too weak. Future improvements could involve adding some geometric checks to filter out this noise before running the optimization.

## 5 Reproducibility Check

- **Code:** Provided in `code/` and documented in the accompanying `README.md`.
- **Data:** Included within project, can be found at `data/`.
- **Environment:** All dependencies specified in `requirements.txt`.
- **Run instructions:** Single-command execution via `bash run.sh`, as documented. See `README.md` for details.
- **Parameters:** Reported values and description of how they were set. Lambda is hard coded on value 1.2.
- **Expected outputs:** Expected graph output values provided in `results/` directory for reproducibility.

## References

- [1] M. Klopschitz, C. Zach, A. Irschara, and D. Schmalstieg, "Generalized detection and merging of loop closures for video sequences," 01 2008.
- [2] T. Schöps, J. L. Schönberger, S. Galliani, T. Sattler, K. Schindler, M. Pollefeys, and A. Geiger, "A multi-view stereo benchmark with high-resolution images and multi-camera videos," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [3] "MechaCritic/Python-Visual-Similarity: A repository for computing similarity between images for image retrieval and unsupervised labelling tasks.."
- [4] P. F. Felzenszwalb and D. P. Huttenlocher, "Efficient belief propagation for early vision," *International Journal of Computer Vision*, vol. 70, pp. 41–54, 2004.